Computer Architecture and System Software

Lecture 09: Memory Hierarchy (cont)

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Important Dates:
  - Final Exam dates announced (Wednesday, April 15th, 6pm-9pm @ **1D62** (Duckworth Gym), details of content to come)
  - As4 will be released on March, 18th
    - Due Wednesday, April 1st at midnight
    - 1 question, 20 points

Recall: Storage Technology Trends

- Observations
  - CPU speed is out-pacing all storage
  - SRAM is keeping up, but
  - DRAM and Disk are sorely lacking
  - SRAM is too expensive to use for main memory
  - DRAM and disk are main storage devices
  - Creates the CPU-memory gap, CPU is limited by speed of memory and disk
  - To bridge gap: modern systems use SRAM based caches
  - This approach works because most programs exhibit good locality

The Memory Hierarchy

6.2 Locality

Locality

- Well written programs tend to reference data items that
  - Are near to other recently referenced items
  - Were recently referenced themselves
- Two forms of locality:
  - Spatial locality: if a memory location was referenced, memory locations nearby will likely be or have been referenced
  - Temporal locality: if a memory location was referenced, the memory location is likely to be referenced again in the near future

Principle of Locality

Why consider locality?

- Programs with good locality will run faster than programs with poor locality
- All levels of a computer system exploit locality:
  - Hardware: uses caches to speed up memory accesses
  - Operating Systems:
    - Main memory is used as a cache for the most recently used chunks of the virtual address space
    - Use main memory as cache for disk data
  - Applications: Web browser cache recently viewed pages on disk

Principle of Locality

- Two different things are fetched from memory during execution
  - Program data: fetched at request of program
  - Program instructions: fetched at request of CPU
- Locality principles apply to both

Locality of Ref. to Program Data

- Consider the following function:

```
int maxvec( int v[N] ) {
  int i, max = 0;

  for( i = 0; i < N; i++ ) {
     if( max < v[i] ) max = v[i];
  }
  return max;
}
```

- Question: Does this function have good locality?

Locality of Ref. to Program Data

- Answer: Look at reference pattern for each variable
- Max:
  - Scalar: no spatial locality
  - Accessed at least once in each loop iteration: good temporal locality
- i:
  - Scalar: no spatial locality
  - Accessed at least once in each loop iteration: good temporal locality
- v:
  - Array elements are accessed sequentially: good spatial locality
  - Each element is accessed once: bad temporal locality

Locality of Ref. to Program Data

- Since all variables have either good temporal or spatial locality
  - maxvec has good locality
- maxvec accesses elements sequentially: said to have a stride of 1
- Definition: stride-k means function visits every kth item
- Stride-1 is most common and has good spatial locality
- Idea: greater the stride, the worse the locality

Locality of Ref. to Program Data

- Multidimensional arrays example (p. 587-588)

- Important: Convention is to store elements of array in *row-major* order

Locality of Instruction Fetches

- CPU fetches instructions from memory in sequential order
- Unless: a branch occurs
- These are also memory accesses
- Small tight loops enjoy both good spatial and temporal locality

```
int sumarraycols (int a[M][N])
{
        int i, j, sum = 0;
        for (j = 0; j <N; j++)
            for (i = 0; i < M; i++)
                sum += a[i][j];
        return sum;
}
```

- Large functions enjoy good spatial locality, bad temporal
- Small functions that are not called from loops enjoy neither

Summary of Locality

- Programs that repeatedly reference the same variables enjoy good temporal locality
- For program with k-stride patterns, the smaller the stride the better the spatial locality
- Loops have good temporal and spatial locality
  - Smaller loops have better spatial locality
  - More loop iterations have better temporal locality

The Memory Hierarchy

6.3 The Memory Hierarchy

The Memory Hierarchy

- We began with two fundamental properties
  - Different storage technologies have widely ranging access times
    - Faster technologies are more expensive and have less capacity
  - Well written programs tend to have good locality
- These principles complement each other
- Suggests organization of computer memory into a hierarchy

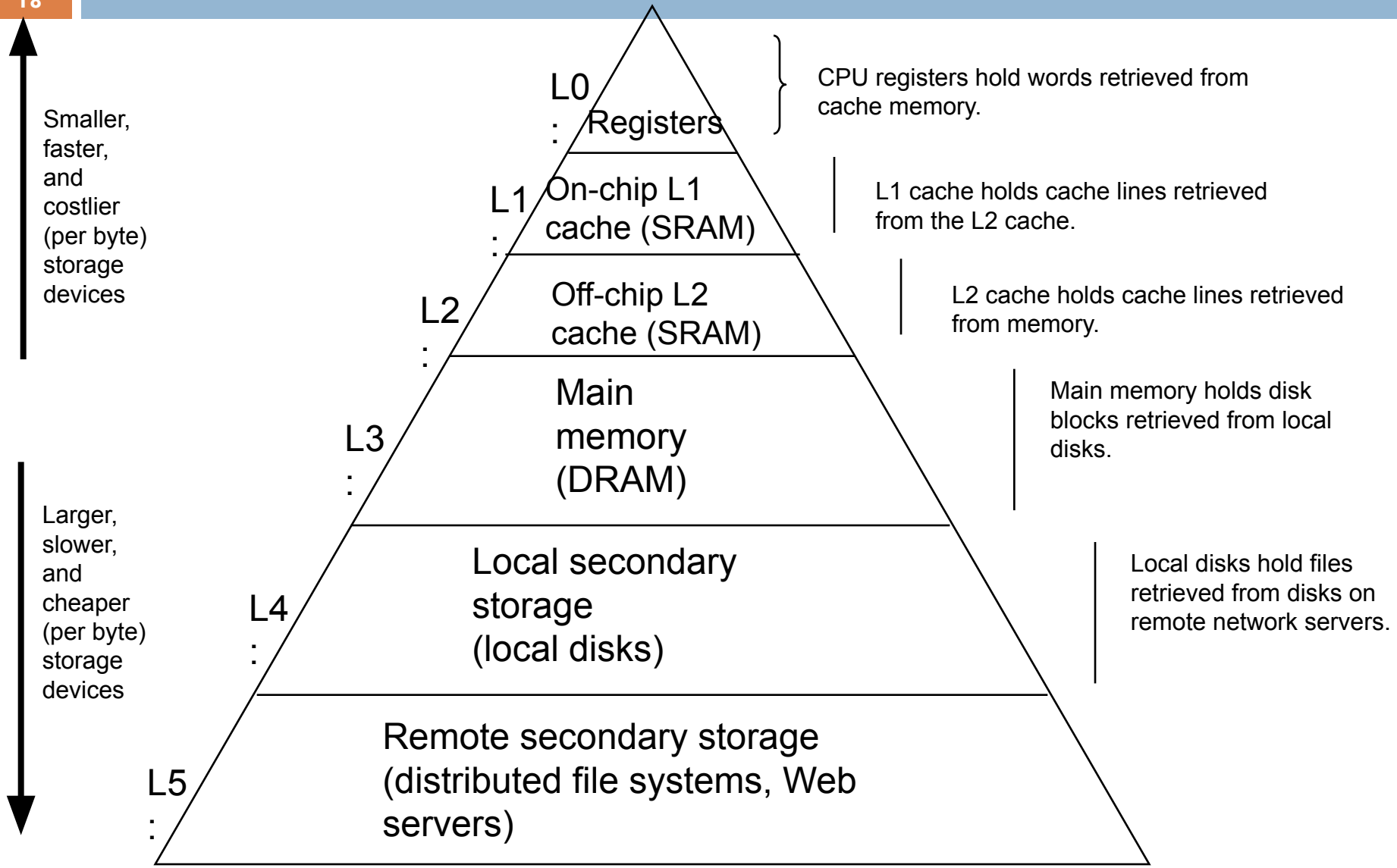
The Memory Hierarchy

- In general:
  - Storage devices get slower, cheaper, and larger as we move from higher to lower levels
  - At the highest level (L0) are a small number of fast CPU registers
    - Accessed in a single clock cycle
  - Next are one or more small SRAM-based cache memories
    - Accessed in a few clock cycles
  - Followed by DRAM-based memory
    - Accessed in 10s to 100s clock cycles
  - Next come slow enormous disks

The Memory Hierarchy

- L0 Registers: Holds words retrieved from L1 cache
- L1 On chip cache: Holds cache lines retrieved from L2 cache
- L2 Off-chip cache: Holds lines retrieved from main memory
- L3 Main Memory: Holds disk blocks retrieved from local disk
- L4 Local Disks: Holds files retrieved from remote sources
- L5: Remote secondary storage: End of hierarchy

Storage Hierarchy

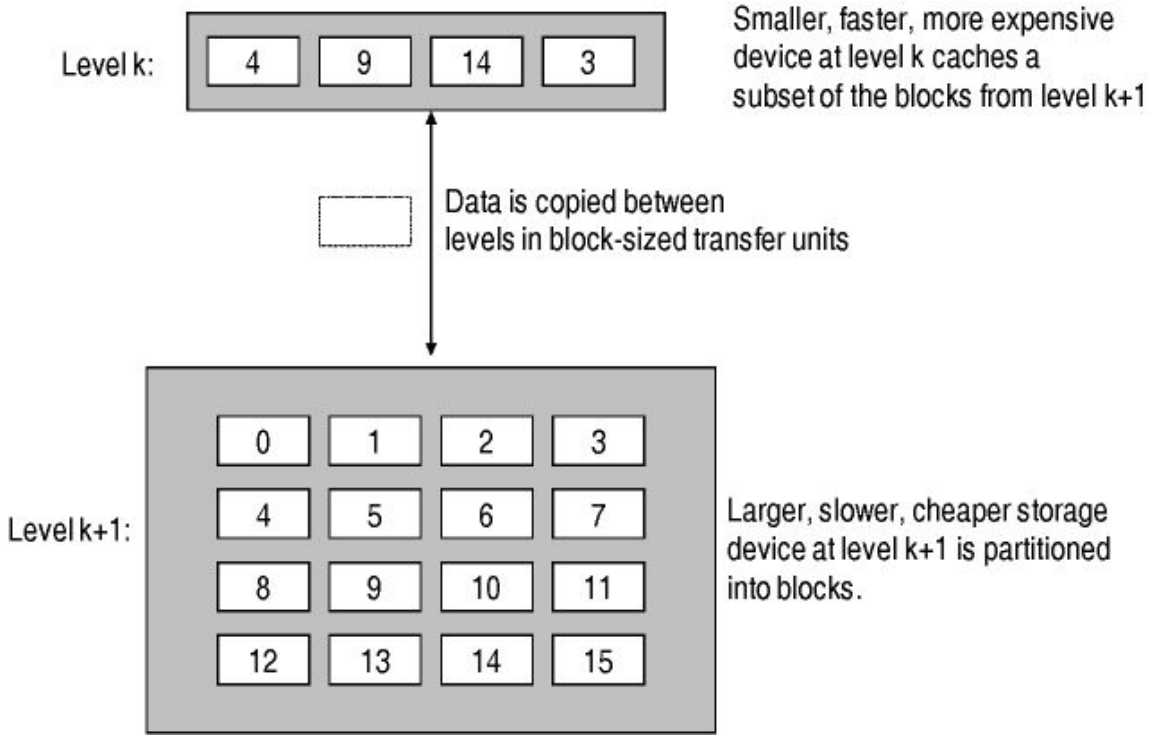


Caching in the Memory Hierarchy

- A cache is a
  - Small fast storage device
  - Acts as a staging area for data objects in slower devices
  - Caching: Process of using a cache
- Main idea:
  - Faster & smaller memory at level k acts as a cache for larger & slower memory at level k+1
    - Each level "caches" data objects from the next lower level

Caching in the Memory Hierarchy

- General concept:
  - Storage at level k+1 is partitioned into blocks
  - Each block has a unique address
  - Blocks can be either fixed (in most cases) or variable size



Caching in the Memory Hierarchy

- General concept continued
  - Storage at level k is partitioned into smaller set of same-sized blocks
  - Data is copied between levels k and k+1 in units corresponding to the size of the block

Level	L0/L1	L1/L2	L2/L3	L3/L4
Size	1 word	4 - 8 words	4 - 8 words	1000 words

  - Note: different block sizes between different levels
  - General principle: lower in hierarchy = longer access and larger blocks

Cache Hits and Misses

- When a program looks for data object d at level k+1
  - It first looks for d at level k
  - If d is cached at level k, then this is called a cache hit
  - Program reads d from level k, which is faster than level k+1

Cache Hits and Misses

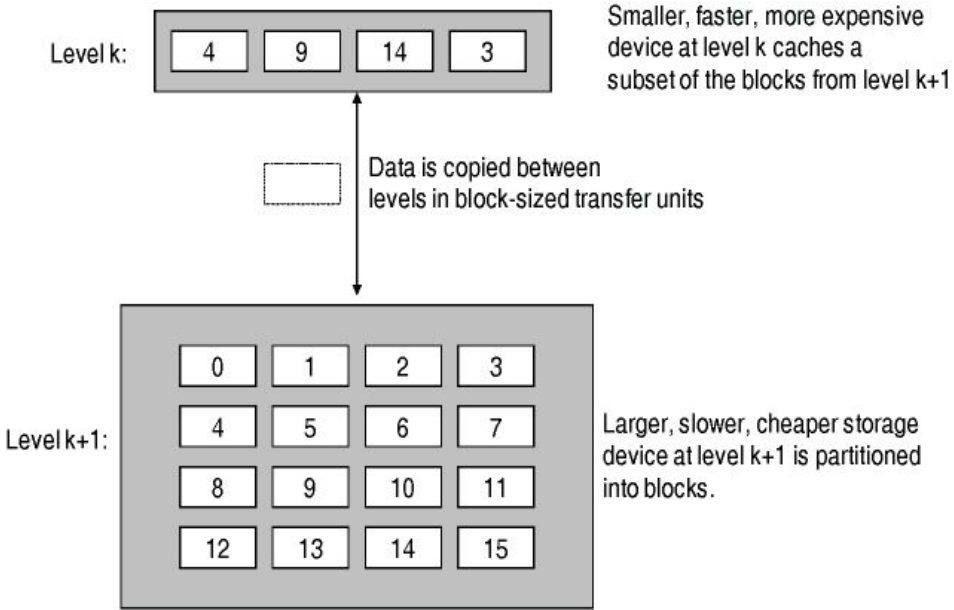
- If d is not found, this is called a cache miss.
  - Cache at level k fetches block d from level k+1
  - If level k cache is full, fetched block overwrites another block in cache
  - The overwritten block is called the victim block
  - The victim block is said to be evicted from the cache
  - Method used to perform eviction is called the replacement policy
    - Random
    - Least Recently Used (LRU)
  - Once d is read into level k, it can be used by the program

Cache Hits and Misses

- Three types of cache misses:
- Compulsory misses: are those misses caused by an empty cache
  - Empty cache is called a cold cache
- Conflict misses: Are those misses that could have been avoided, had the cache not evicted an entry earlier
- Capacity misses: Misses that occur solely due to finite size of the caches
- When a block is loaded into cache, it must have a place
- Ideal: a flexible policy to place block anywhere in cache

Cache Hits and Misses

- Problem: caches at top of hierarchy must be fast, such a policy would be too expensive to implement in hardware
- Solution:
  - Hardware caches restrict where blocks can be placed
  - To a subset or even singleton of blocks at level k
  - Example: block i can be placed only in location i mod 4



Cache Hits and Misses

- However:
  - Even if cache is not full, another block may have to be evicted
- Lastly:
  - Programs generally work in phases (or stages)
  - In each stage, program access a limited # of blocks
  - This set of blocks is called the working set
  - If working sets fits in cache, great! Program runs quickly
  - If working set does not fit, program wastes time evicting and replacing blocks in cache

Cache Management

- At each level of memory something must manage the cache
- *i.e.* evict and load blocks, and decide which blocks to replace
- This logic can be hardware, software, or both
  - Compiler manages L0
  - Hardware manages L1/L2
  - Hardware/OS manages L3
  - OS manages L4 (many disks also have a hardware cache)

Summary of Memory Concepts

- Exploiting Temporal Locality:
  - Objects will be accessed many times
  - First time object is loaded into cache
  - In the future object is accessed from the cache faster
- Exploiting Spatial Locality:
  - Blocks contain multiple data objects
  - First object causes block to be loaded into cache
  - Next object accessed after first object will already be in the cache

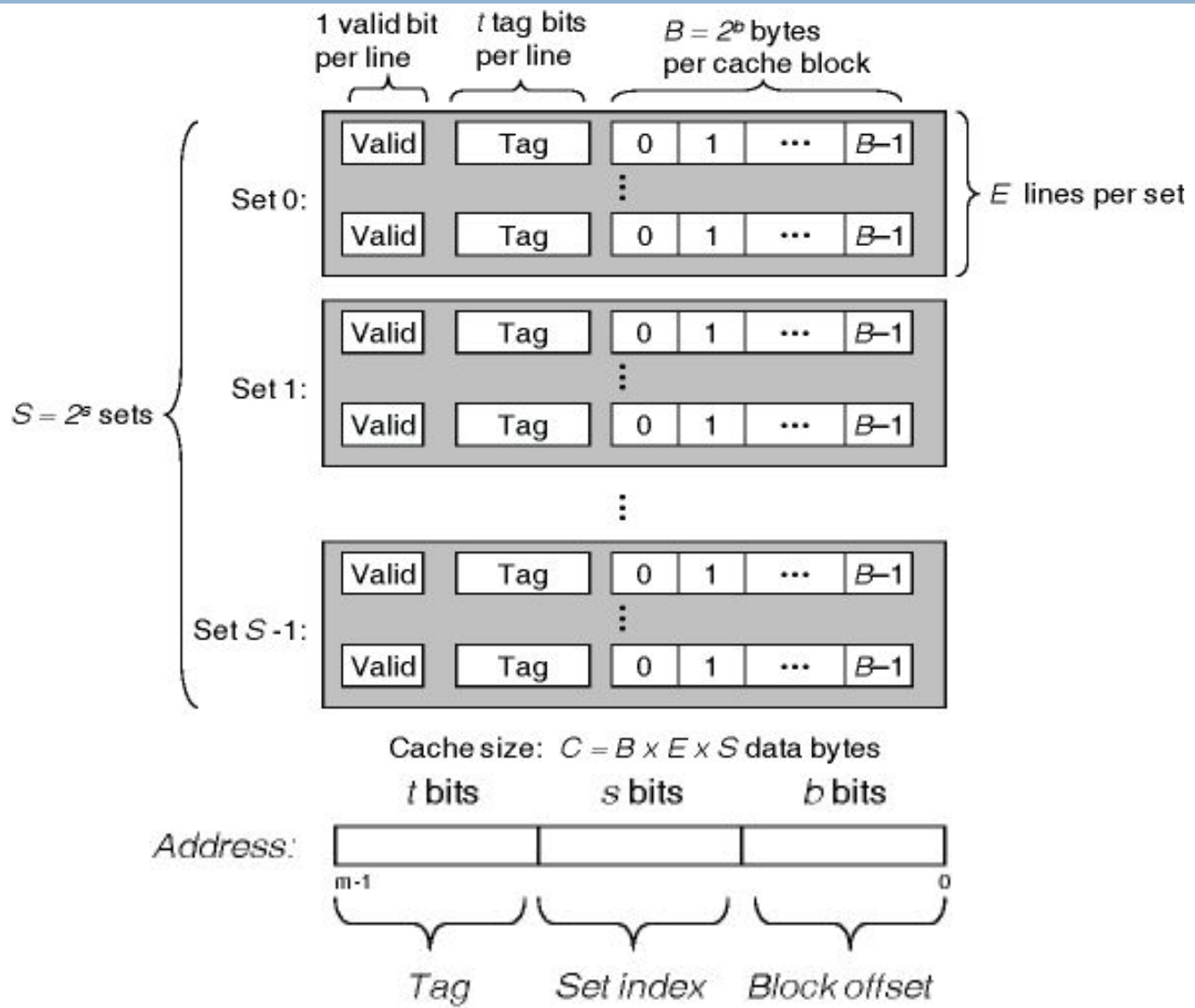
The Memory Hierarchy

6.4 Cache Memories

General Operation

- Assume the following memory hierarchy
  - Registers
  - L1 cache
  - Memory

Generic Cache Structure



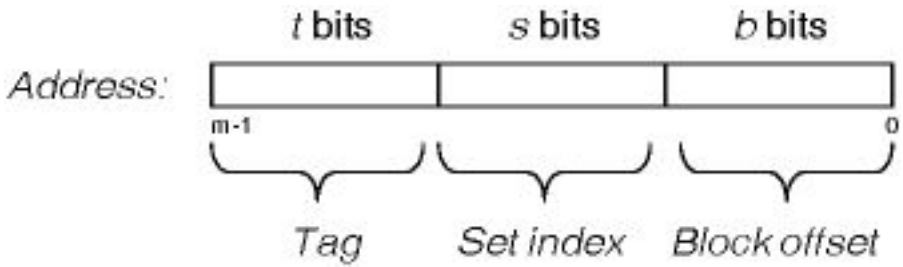
Summary Of Cache Parameters

Fundamental Parameters	
Parameter	Description
$S = 2^s$	Number of sets
E	Number of lines per set
$B = 2^b$	Block size (bytes)
$m = \log_2(M)$	Number of physical (main memory) address bits

Derived Quantities	
Parameter	Description
$M = 2^m$	Maximum number of unique addresses
$s = \log_2(S)$	Number of set index bits
$b = \log_2(B)$	Number of block offset bits
$t = m - (s + b)$	Number of tag bits
$C = B \times E \times S$	Cache size (bytes) not including overhead such as the valid and tag bits

General Operation

- CPU requests word at address A (*i.e.* data is not in the reg.)
- Request is sent to cache
- A is divided into three parts
  - Set: used to determine which set the block may be cached in
  - Tag: used to determine which line, if any, the block is in
  - Offset: offset of word in block



General Operation

- Cache uses *s* bits of A to identify the set that may contain the block
- Cache uses tag and valid bit to determine if a line in set contains the block
- Offset bits are used to load word if found
- Otherwise cache loads block from memory

General Operation

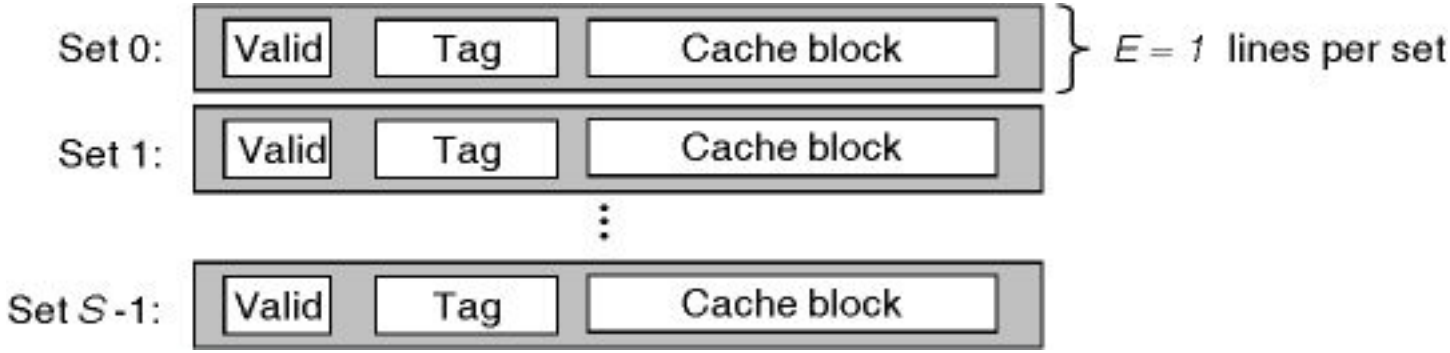
- The cache must
  - determine whether a request is a hit or a miss, and
  - extract the requested word.
- This process consists of three steps:
  1. Set selection
  2. Line matching
  3. Word extraction

Types of Cache

- Caches are grouped into different classes based on E (# of cache lines per set)
  - Direct-mapped caches: easiest to understand and implement
  - Set associative caches: hard to implement
  - Fully associative caches: hard to implement

Direct-Mapped Cache

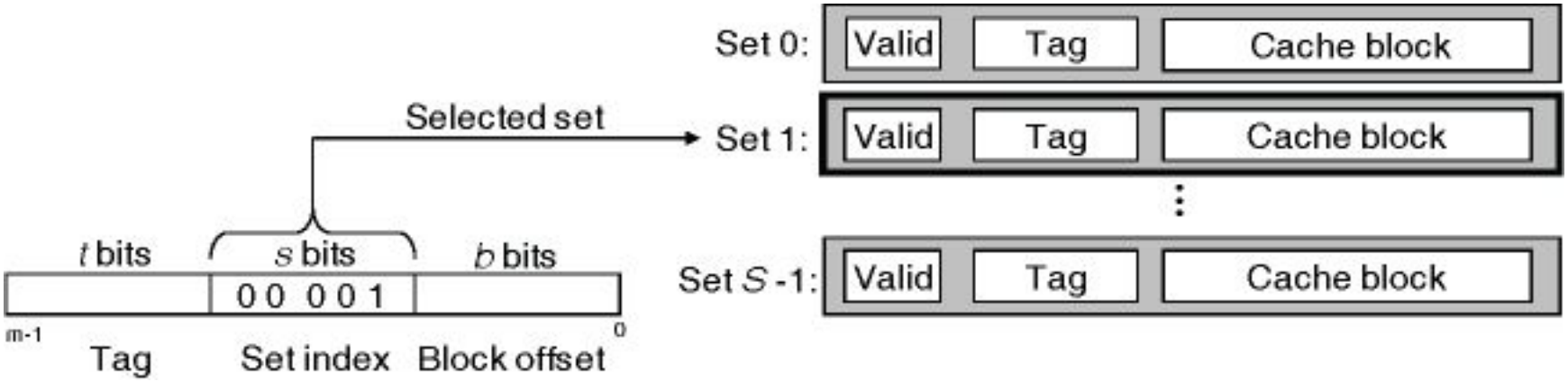
- Key characteristic: each set has 1 line (*i.e.* E = 1)
- Therefore # of sets = # of lines



DM: Set Selection

To determine if cache contains word at address A

- Find set: use s bits of A to index into array of sets

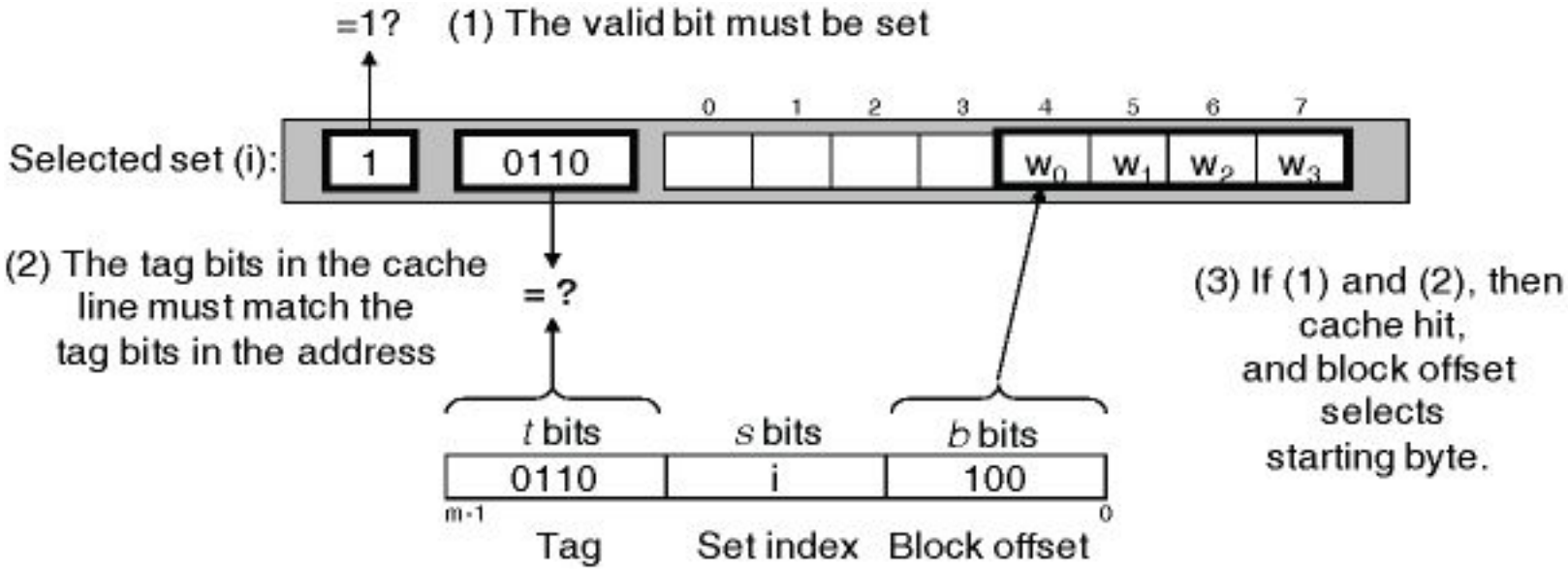


DM: Line Matching

- Check if valid bit is set
- Check if tag bits of A match tag of line
- If the above conditions are true, then we have a cache hit
- Otherwise, we have a miss.
  - Load block from memory (assuming only one cache)
  - Replace line with block
  - Set bit to valid
  - Extract word in block

DM: Word Selection

- When a hit occurs (or block was loaded from memory) we know that word is somewhere in the block
- The block offset provides us with the offset of the first byte in the desired word
  - Think of block as an array of bytes, and the byte offset as an index into that array



Example (pg. 619-621)

- The mechanisms that a cache uses to select sets and identify lines are extremely simple
  - Have to be, because hardware must perform them in a few nanoseconds
- However, manipulating these bits can be confusing to us

Example (pg. 619-621)

- Let (S,E,B,m) = (4, 1, 2, 4)

	Address Bits			
Address (decimal)	Tag Bits ($t = 1$)	Index Bits ($s = 2$)	Offset Bits ($b = 1$)	Block Number (decimal)
0	0	00	0	0
1	0	00	1	0
2	0	01	0	1
3	0	01	1	1
4	0	10	0	2
5	0	10	1	2
6	0	11	0	3
7	0	11	1	3
8	1	00	0	4
9	1	00	1	4
10	1	01	0	5
11	1	01	1	5
12	1	10	0	6
13	1	10	1	6
14	1	11	0	7
15	1	11	1	7

Example (pg. 619-621)

- Using the table, if I try to access address 8,
  - tag = 1, set = 0, block offset = 0
- A read at address 8 therefore checks the cache at set 0 for a tag of 1. If found, the byte at offset 0 is read (Cache Hit)
- Otherwise, update the cache:

Set 0:	1	1	M[8]	M[9]

- (M[8], M[9] are the bytes of data located at addresses 8, 9)

Example (pg. 619-621)

Set 0:	1	1	M[8] M[9]

- Now try to access address 0. Using the table,
  - tag = 0, set = 0, block offset = 0
- Currently, the cache at set 0 shows tag of 1. We wanted a tag of 0, so this is a cache miss.
- Evict the line and update the cache:

Set 0:	1	0	M[0] M[1]

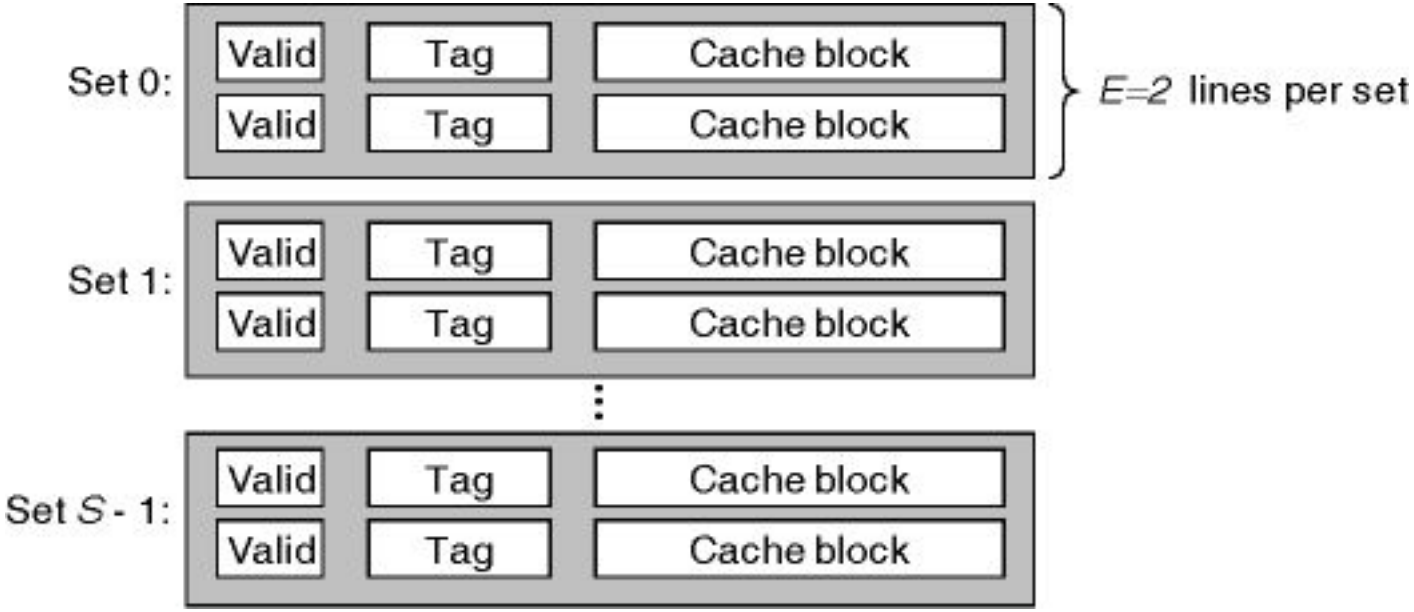
- (M[0], M[1] are the bytes of data located at addresses 0, 1)

Direct Mapped Cache

- Advantage: very fast
  - Code to determine if set contains block is very simple
- Disadvantage: each set can only hold one line
  - Results in thrashing
- Example of thrashing (occurred in last example)
  - Two blocks map to the same set
  - Program accesses each block in alternating order
  - Each time block is accessed, other block is evicted from cache
  - Cache is forced to reload block on each access
  - Slows down program execution significantly (as much as 2x or 3x)
- Also, conflict misses stem from the constraint that each set has exactly one line

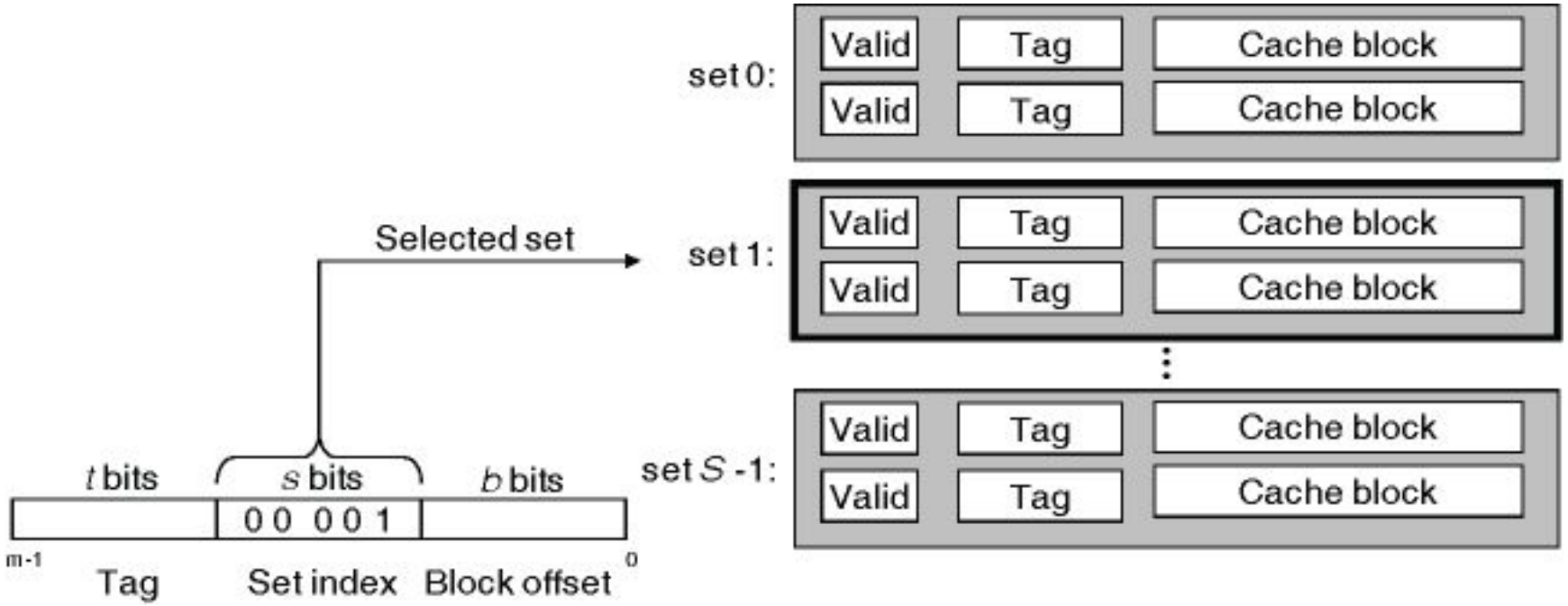
Set Associative Caches

- Key characteristic: 1 < E < C/B
- Called E-way set associative caches
- Each set contains multiple lines



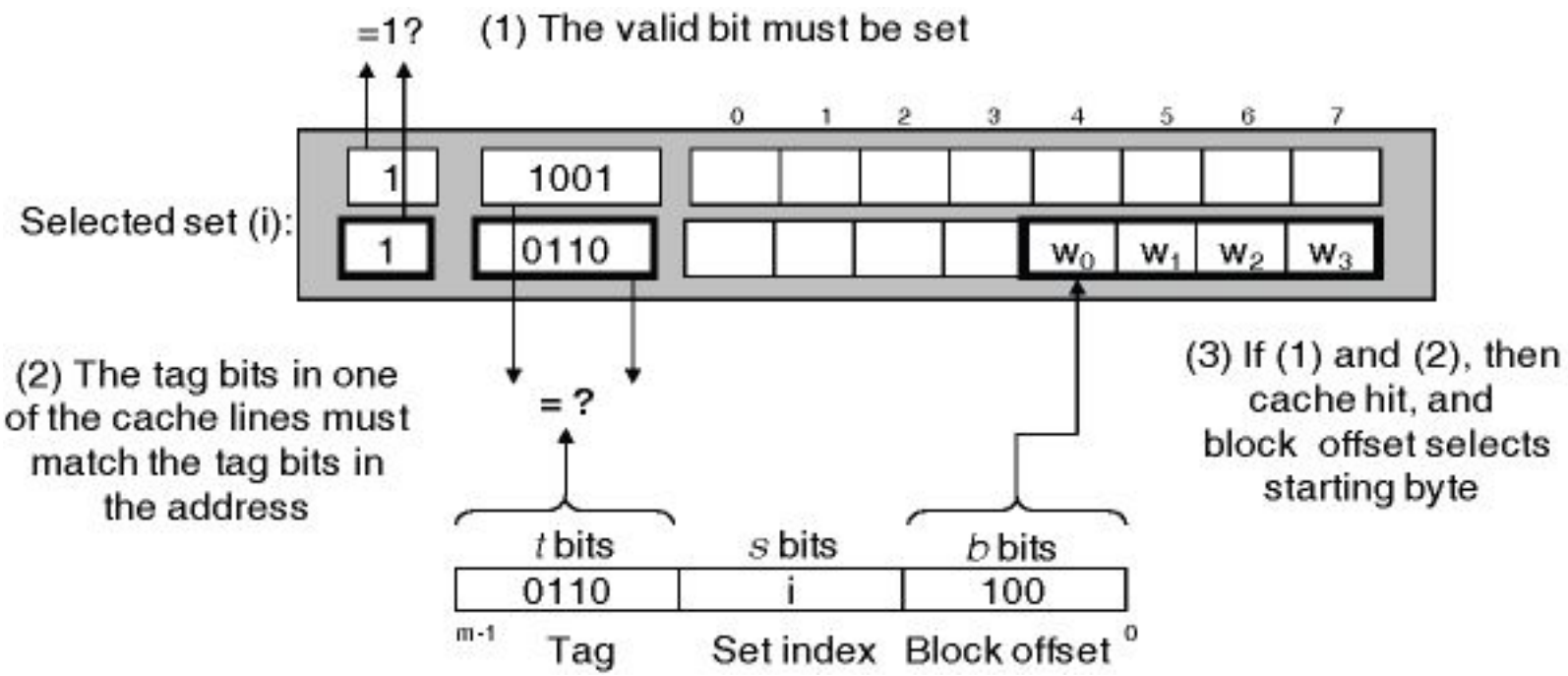
SA: Set Selection

- Identical to direct-mapped caches
  - s bits identify the set



SA: Line Matching and Word Sel.

- Check ALL lines in set (in parallel)



Set Associative Caches

- Retrieve word if line is valid and if line contains matching tag
- Otherwise, load block from memory (assuming only 1 cache)
  - If no empty lines are available (all lines are valid), then evict a line from the set
  - Use offset to get word in cached block

Line Replacement

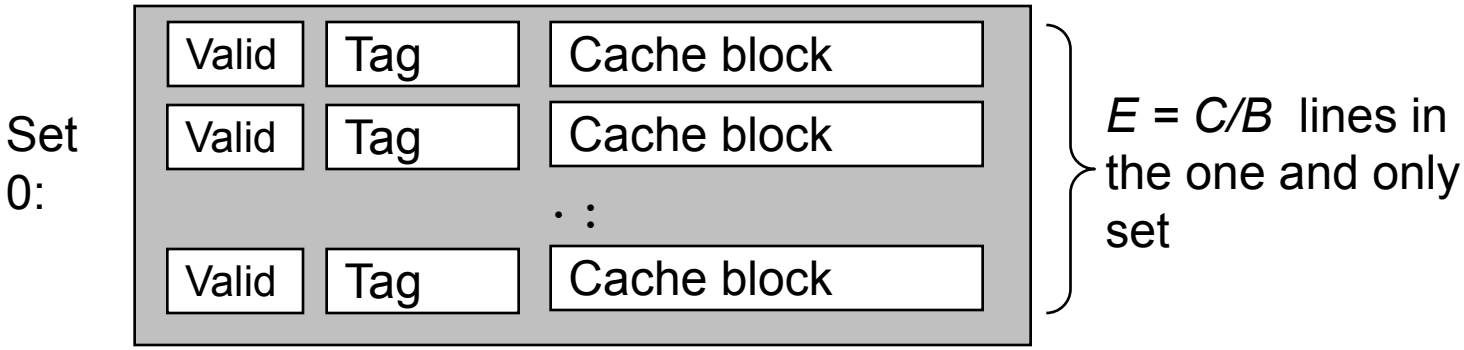
- Which line to evict?
- Replacement policy: Method to select block for eviction
- Options:
  - Random: choose a line at random from the set
  - LFU: Least frequently used
  - LRU: Least recently used
- First policy is cheap, but results in more conflict misses
- Latter policies are more expensive, but result in less conflict misses

Caching Example

- General:
  - Same procedure as previous example but now we have 2 lines of space for each cache set
  - If both lines are full, choose which line to evict based on a replacement policy
  - Replacement policy will vary depending on the storage medium
  - More on replacement policies later

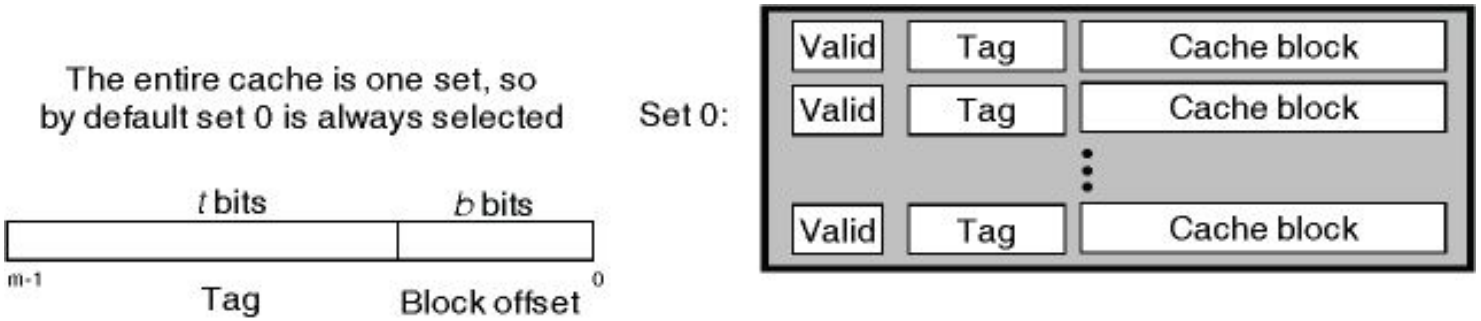
Fully Associative Caches

- Key Characteristic: E = C/B (S = 1)
- Cache is a single set with C / B lines
- Address is divided into tag and offset
  - No s bits.
- Analogous to a huge hash table

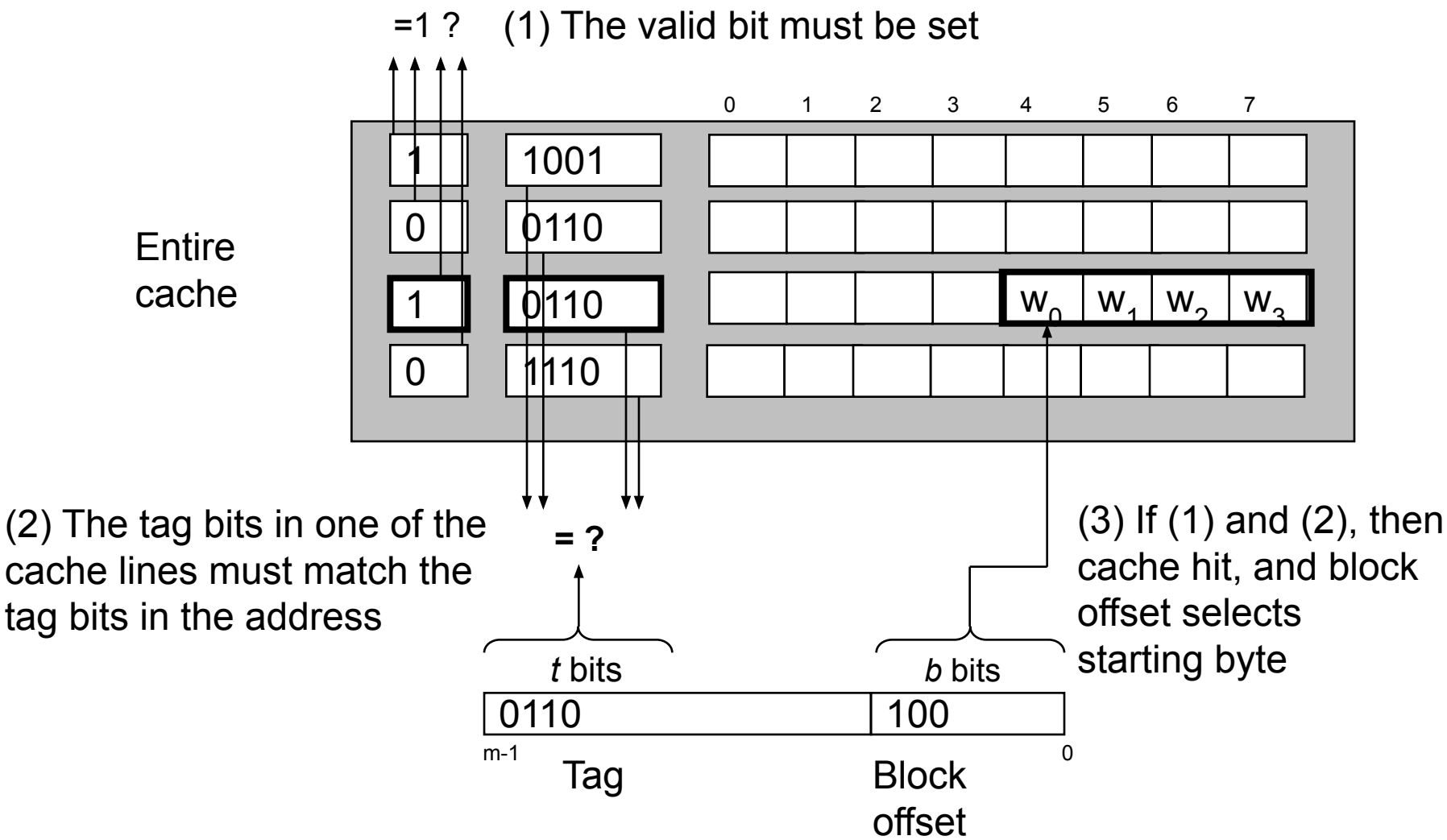


Fully Associative Caches

- Works similar to set associative caches
  - There is only one set
- Check all lines in set (in parallel)
- Retrieve word if line is valid and line contains matching tag
- Otherwise:
  - Choose empty line to place block
  - Or, evict a block if there are no empty lines
  - Load block from memory (assuming only 1 cache)
  - Use offset to get word in cached block



FA: Line Matching and Word Sel.



Fully Associative Caches

- Logic for searching for tags is slow and expensive
- Only an option in caches at lower end of hierarchy
- Too slow for L1 and L2 cache
- L1 and L2 caches use either
  - Direct mapped caches
  - 2-way caches
  - 3-way caches
  - 4-way caches

Caches and Memory Writes

- What about writing to memory?
- Recall read procedure:
  - CPU requests word from cache
  - If block with word is cached, it's a hit
  - Else it's a miss, and cache fetches block from next level
  - Word from block is returned once block is cached

Caches and Memory Writes

- Writes are more complicated
- Scenario: CPU writes a word to memory
- Either block with word is in cache, or not
- If block is in cache (cache hit):
  - Block in cache is updated with word
  - Eventually memory has to be updated with word
    - What does cache do about updating the copy of w in the next lower level in the hierarchy?

Caches and Memory Writes

Two options:

- Write-through: Immediately write block to memory
  - Simplest to implement
  - Increases number of bus transactions
- Write-back: Defer block write until block is evicted
  - Advantage: significantly reduces number of bus transactions
  - Disadvantage: additional complexity
    - Cache must maintain a 'dirty bit' to keep track of which blocks must be written back when evicted
  - Loading cache may take longer because eviction is more complex

Caches and Memory Writes

- Scenario: CPU writes a word to memory
- Either block with word is in cache, or not
- If block is not in cache (cache miss)
  - Should the block be loaded?

Caches and Memory Writes

Two options

- Write-allocate: Update cache only
  - Exploits spatial locality of writes
  - Reduces # of bus transactions
  - Generally done by write-back caches
  - Requires more cache hardware
- No-write allocate: Send update only to lower level
  - More bus transactions will occur
  - Cache updated only on hits
  - Generally done by write-through caches
  - Takes less hardware

Types of Caches

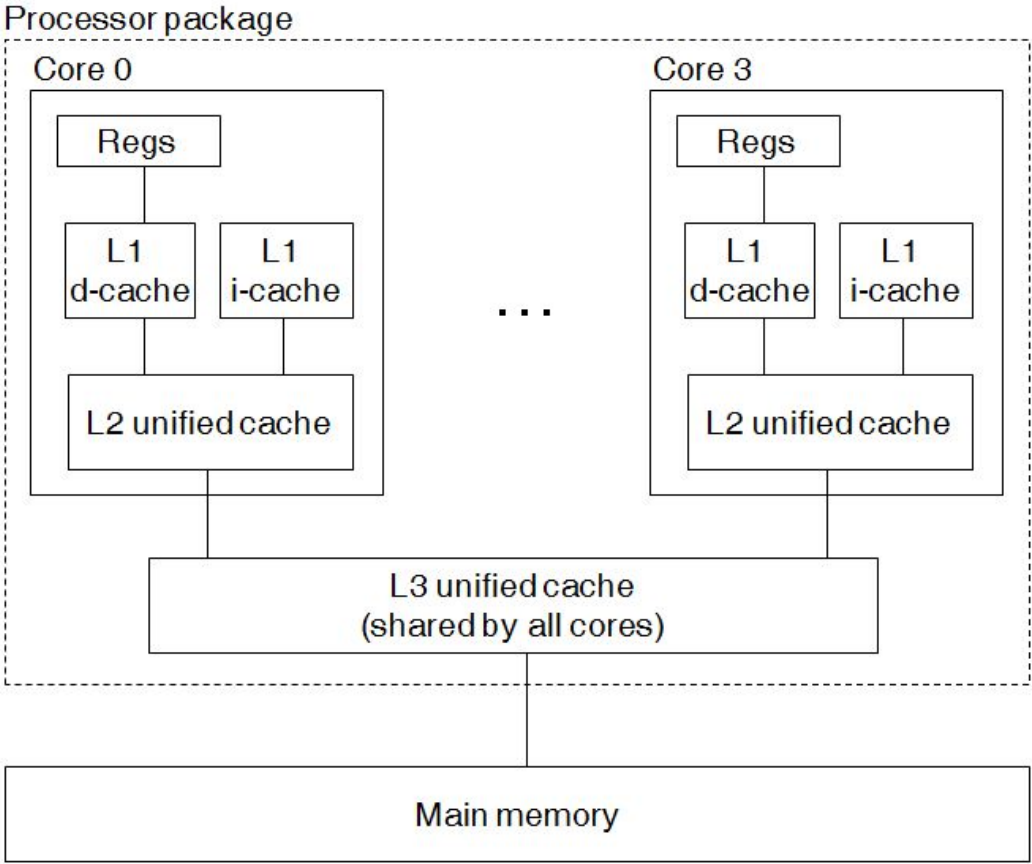
- In many cases CPU'S use two caches:
  - d-cache: for program data
    - Should handle a wide variety of access patterns
    - Handles reads/writes
  - i-cache: for program instructions
    - Mainly needs to handle simple sequential access
    - Does not need to handle writes
    - Can be made simpler and faster than a d-cache
- Unified cache: a single cache is used for both instructions and data

Types of Caches

- Modern processors include separate i-caches and d-caches
  - Processor can read an instruction word and a data word at the same time
  - I-caches are typically read-only (simpler)
  - Each cache is often optimized to different access patterns
    - Different block sizes, associativities, and capacities

Types of Caches

- Cache hierarchy for the Intel Core i7 processor
- Each CPU has four cores
- Each core has its own private L1 i-cache and L1 d-cache
- Each core also has its own L2 unified cache
- All of the cores share an on-chip L3 unified cached
- Note: all SRAM cache memories are contained on the chip



Performance Impact

- Cache performance is evaluated using several metrics
- Miss rate: fraction of memory references that are cache misses
  - # misses / # references
- Hit rate: fraction of memory references that are cache hits
  - 1 – miss rate
- Hit time: Time to deliver a cached word to the CPU
  - Including time for set selection, line identification, and word selection
  - several cycles for L1
- Miss penalty: Additional time required due to a miss
  - 10 cycles to load from L2
  - 40 cycles to load from L3
  - 100 cycles to load from memory

Parameters and Performance

- Recall, cache parameters are
  - Cache size: # of bytes cache can store
  - Block size: # of bytes stored in a line
  - Associatively: # of lines per set
- Impact of cache size
  - Adv.: Large caches tend to increase hit rate
  - Disadv.: Large caches tend to increase hit time
    - Especially important for L1 caches that must have short hit time

Parameters and Performance

- Impact of block size
  - Large blocks can increase hit rate if spatial locality is good
  - Large blocks imply smaller number of cache lines (C = SxExB)
    - Reduction in hit rate in programs with good temporal locality
  - Large blocks increase miss penalty (time to load blocks)
    - Since larger blocks cause larger transfer times
  - Modern systems usually compromise
    - Blocks that contain 32 to 64 bytes

Parameters and Performance

- Impact of associatively (the number of lines E per set)
- Advantage of higher associatively
  - Reduces thrashing due to conflict misses
- Disadvantages:
  - Slower and more expensive to implement
  - Hard to make fast
  - Requires more tag bits
  - More bits to keep track of which block to evict next
  - Can increase hit time because of increased complexity
  - Can increase miss penalty because of increased complexity of choosing a victim
- Essentially a trade-off between cost, hit-rate, and miss penalty

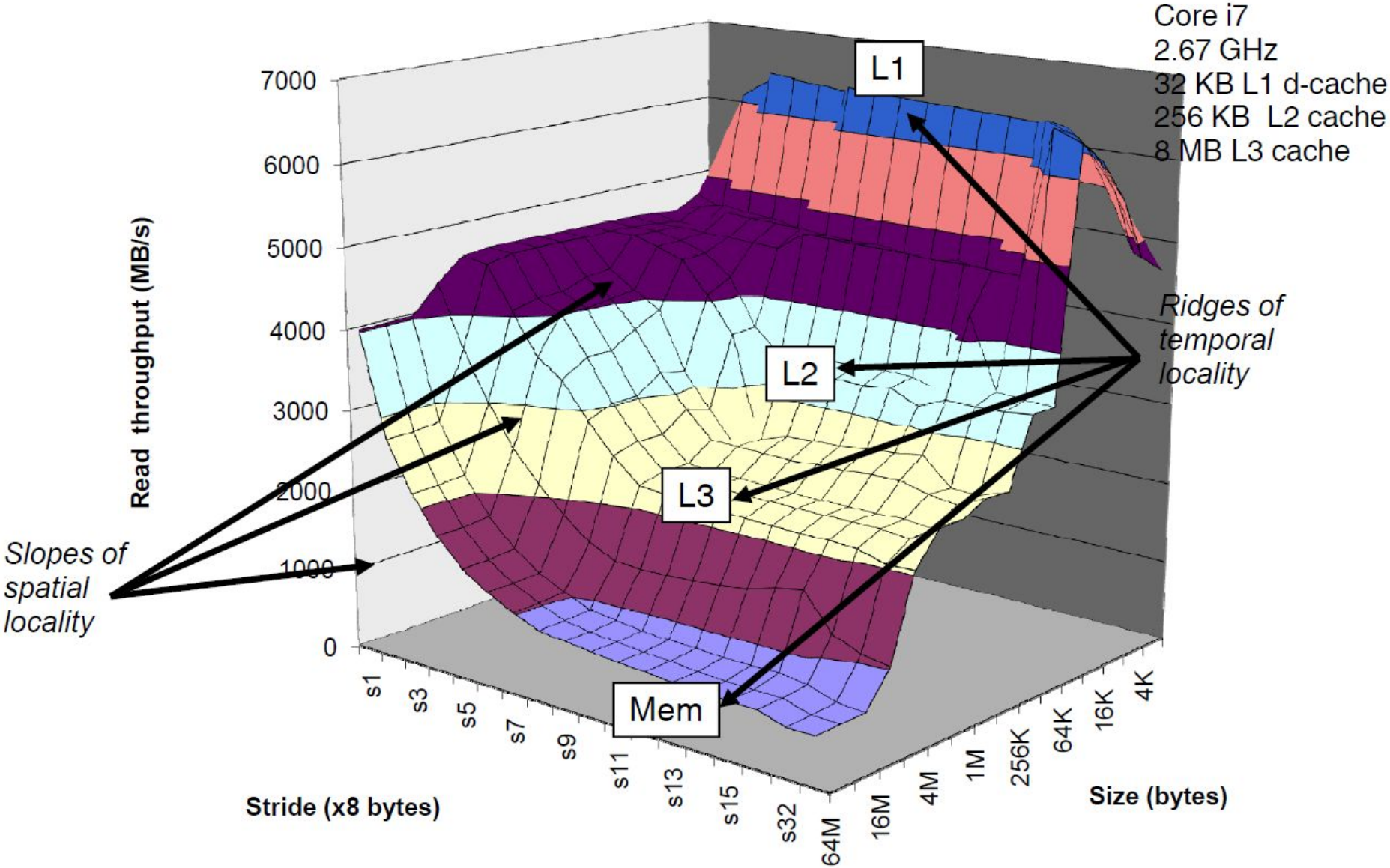
Parameters and Performance

- Write-through caches are simpler to implement
  - Can use a write buffer that works independently of cache to update memory
- Read misses are less expensive
  - Do not trigger a memory write
- Write-back caches result in fewer transfers
  - Allows more bandwidth to memory for I/O devices
  - Reducing the # of transfers becomes important as we move down the hierarchy
- In general, caches further down the hierarchy are more likely to use write-back caches

Memory Mountain

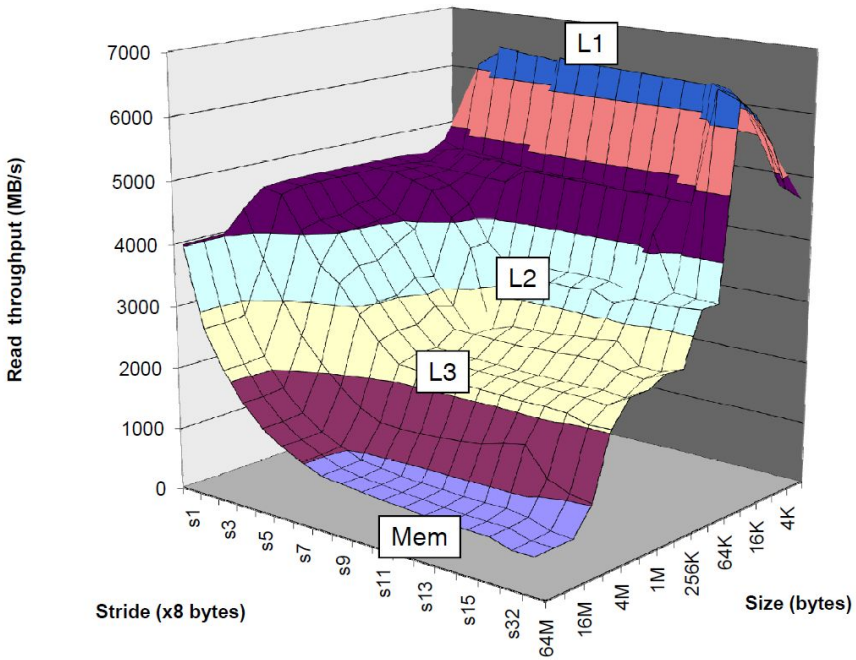
The Memory Mountain

- Every computer has a unique memory mountain
  - Characterizes the capabilities of the memory system
- Next slide shows the memory mountain for an Intel Core i7 system
  - L1 cache: 32KB
  - L2 cache: 256KB
  - L3 cache: 8MB
- Working set:
  - size varies from 2 KB – 64 MB
  - stride varies from 1 to 64 elements


Core i7
2.67 GHz
32 KB L1 d-cache
256 KB L2 cache
8 MB L3 cache
L1
L2
L3
Mem
Ridges of temporal locality
Slopes of spatial locality
Read throughput (MB/s)
7000
6000
5000
4000
3000
2000
1000
0
Stride (x8 bytes)
s1
s3
s5
s7
s9
s11
s13
s15
s32
Size (bytes)
64M
16M
4M
1M
256K
64K
16K
4K

The Memory Mountain

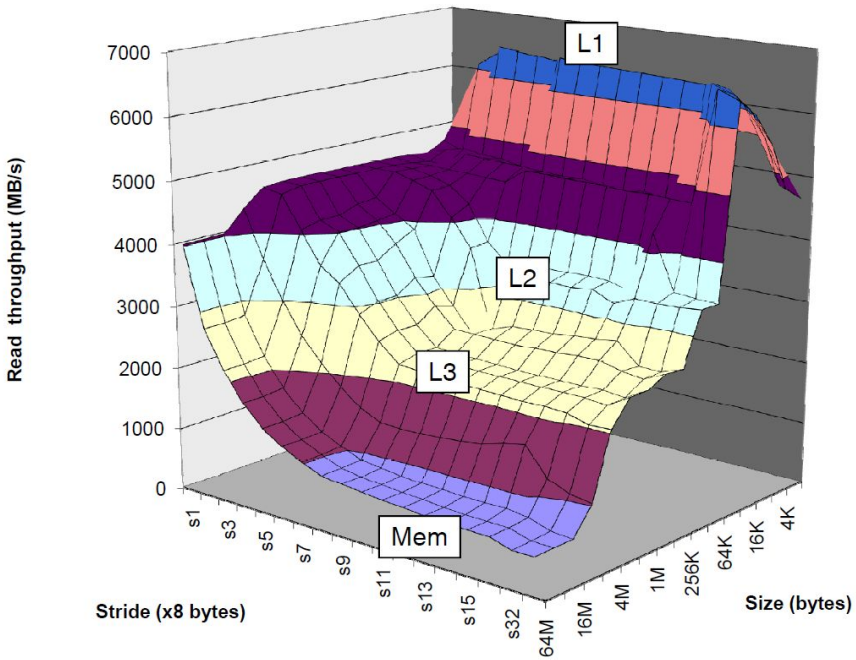
- Geography reveals a rich structure
- Perp. to the size axis are four ridges
  - Correspond to regions of temporal locality
    - *i.e.* working set fits entirely
- Note: order of magnitude difference between top of L1 ridge and bottom of memory ridge
  - Reads at 6 GB/s vs 600 MB/s



The Memory Mountain

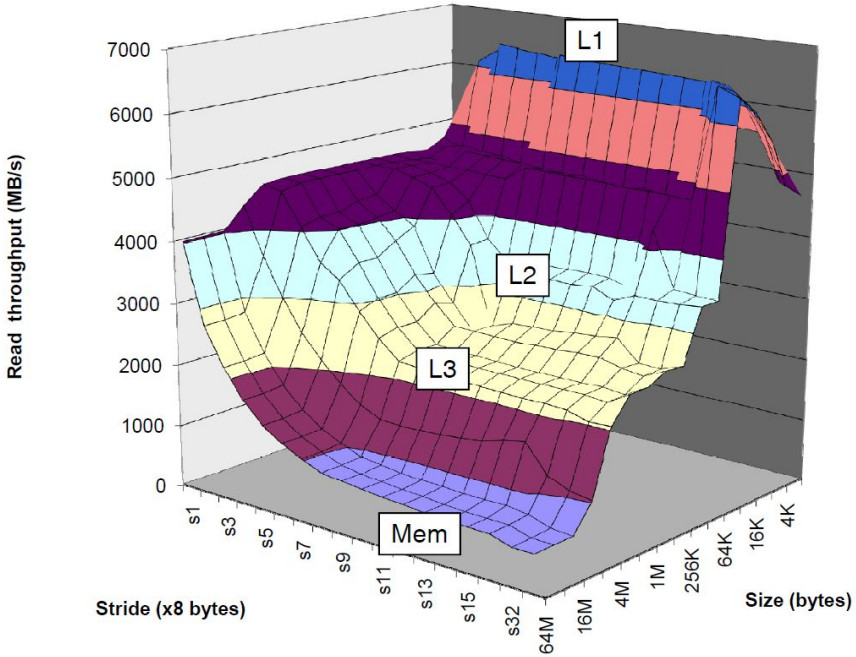
- For L2, L3, and main memory ridges there is a slope of spatial locality that falls downhill as stride increases
  - Increase in stride = decrease in locality
- Notice even when the working set is too large to fit in any of the caches, the highest point on the main memory ridge is a factor of 7 higher than its lowest point
  - Even with poor temporal locality, spatial

Locality can still come to the rescue



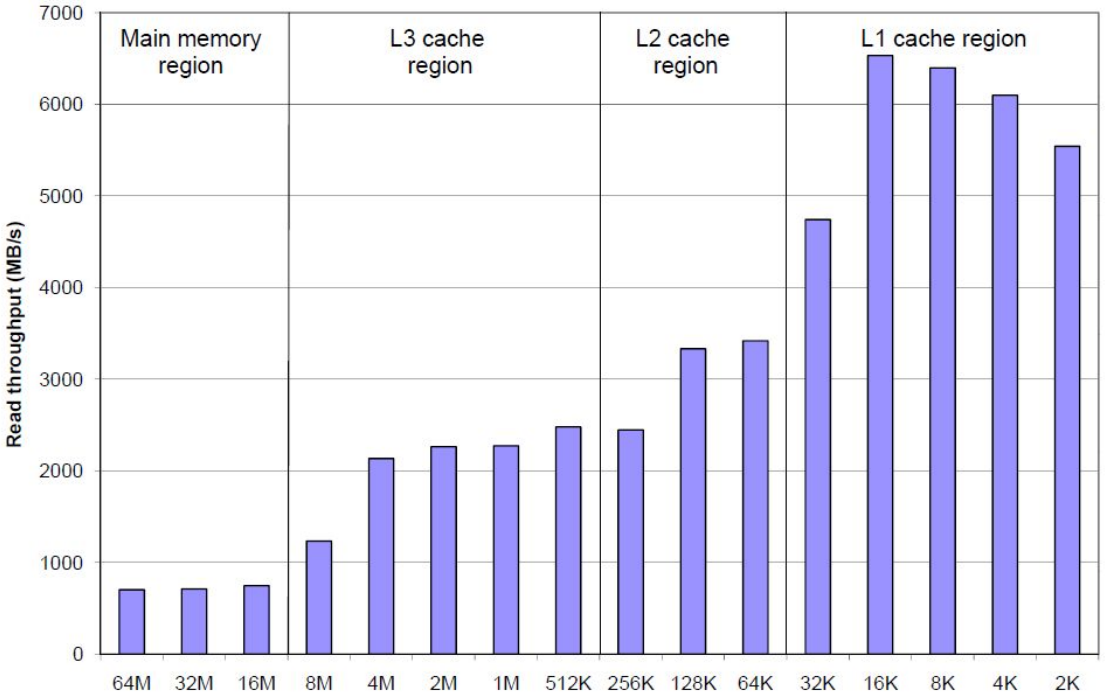
The Memory Mountain

- Notice flat ridge for stride 1
  - Read throughput is relatively constant at 4.5 GB/s
- Due to prefetching mechanism in the Core i7 memory system
  - Automatically identifies memory referencing patterns and attempts to fetch those blocks into cache before they are accessed
  - Yet another reason to favor sequential access in your code



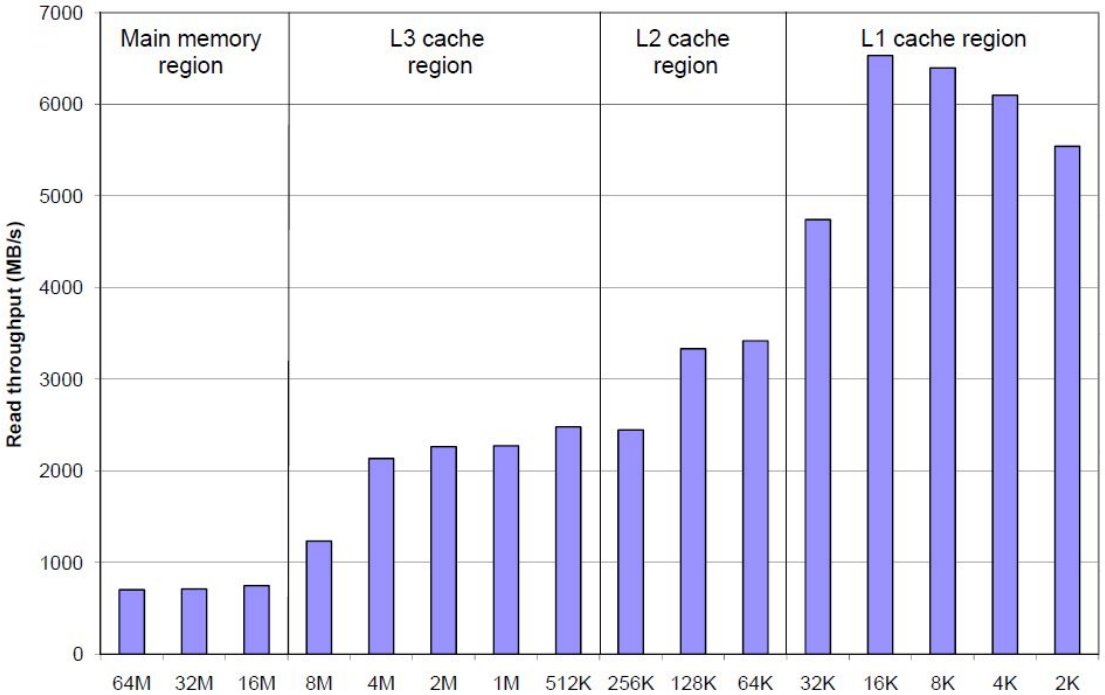
The Memory Mountain

- Let's take a slice of mountain holding stride constant
  - To see impact of cache size and temporal locality on performance
- Up to 32 KB, working set fits entirely in L1 d-cache
  - Thus, reads are served at the peak throughput (6 GB/s)



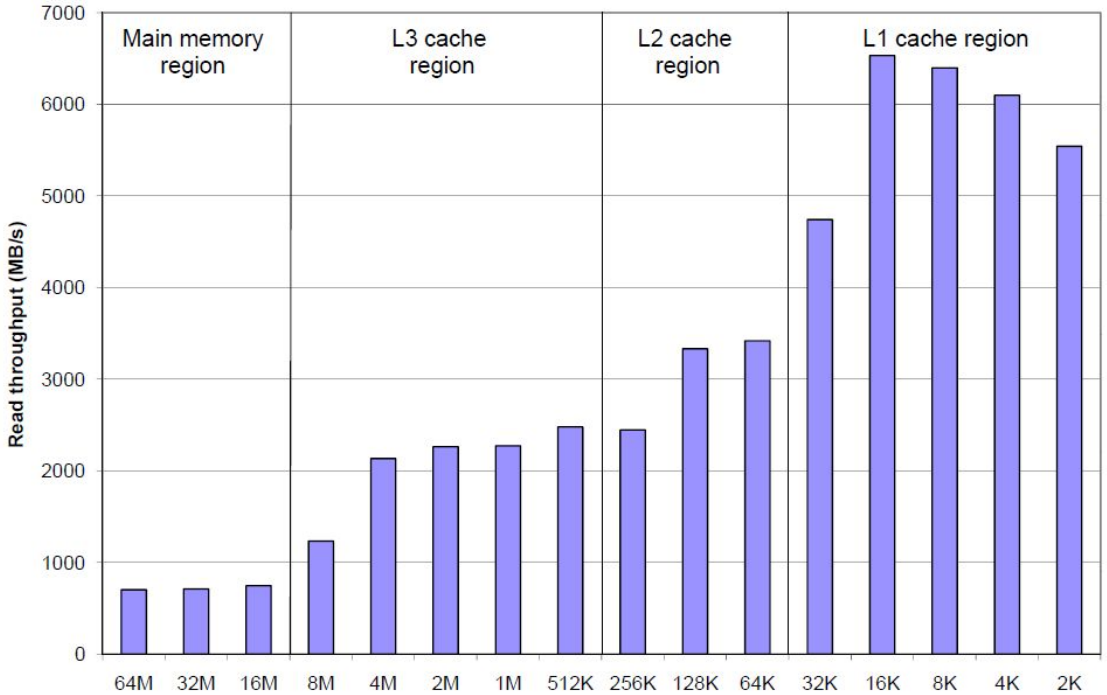
The Memory Mountain

- Up to 256 KB, working set fits entirely in L2 cache
- Up to 8M, working sets fits entirely in L3 cache
- Larger working sets are served from memory



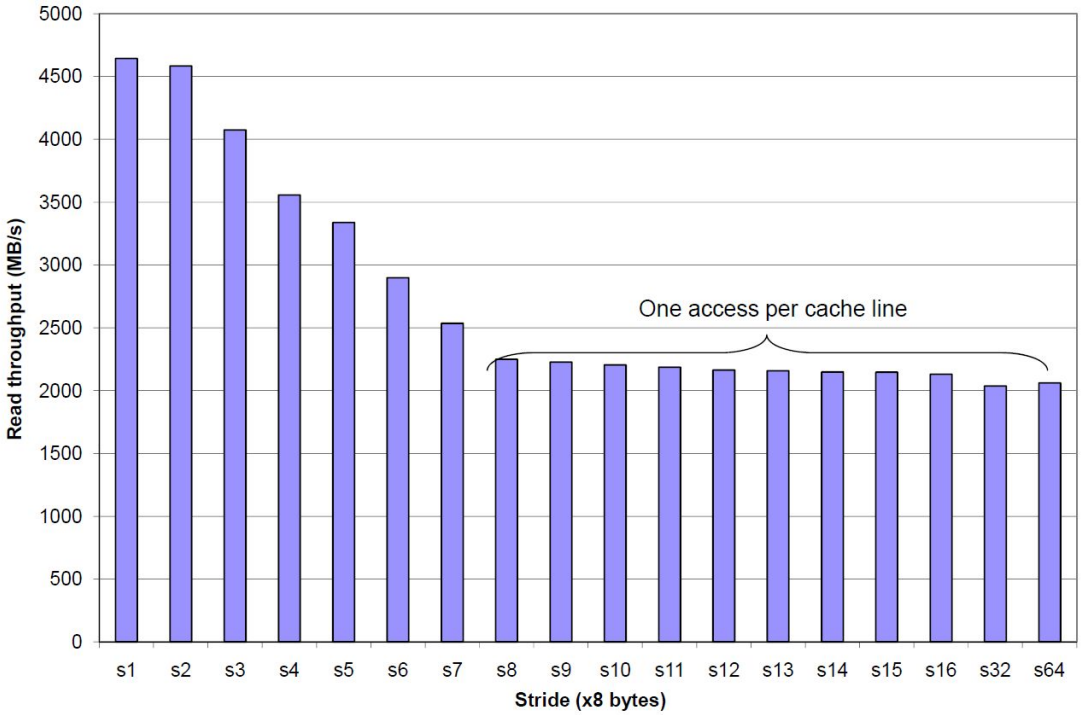
The Memory Mountain

- Notice that read throughputs drop when the working sets are equal to their respective cache sizes
  - Likely drops are caused by other data and code blocks that make it impossible to fit the entire array in the cache



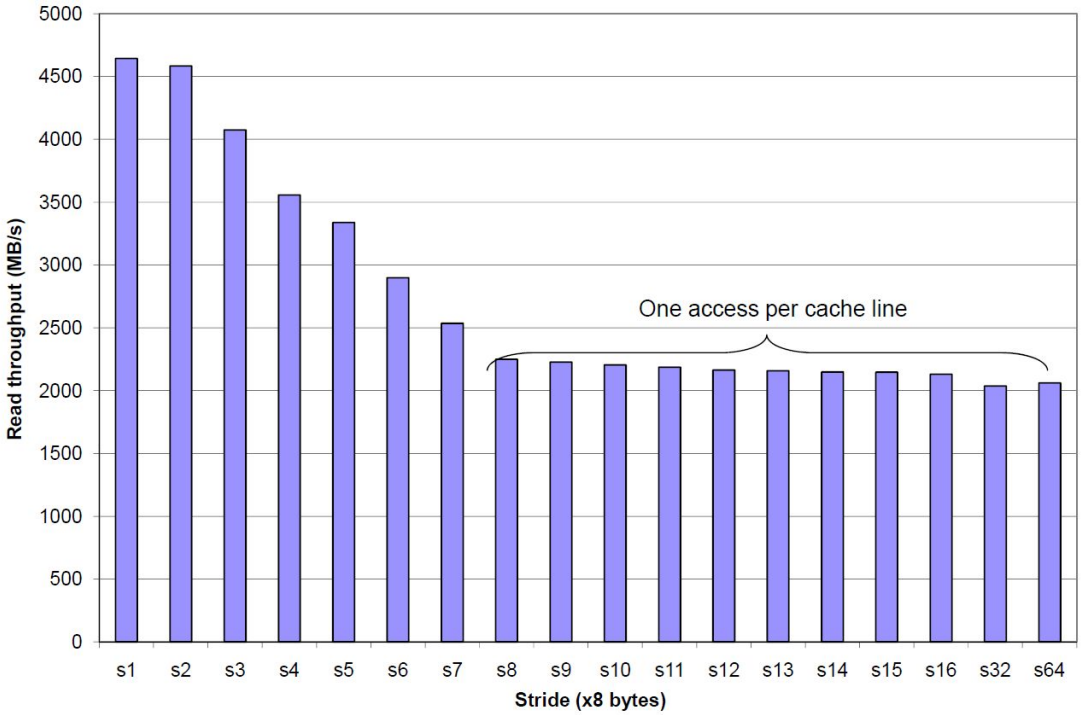
The Memory Mountain

- Let's take a slice of the mountain holding working set size constant
  - To see impact of spatial locality on read throughput
  - Let's use a fixed size of 4 MB
    - Cut along L3 ridge
    - Working set fits entirely in L3 cache (too large for L2 cache)



The Memory Mountain

- Notice read throughput decreases steadily as the stride increases from 1 to 8
  - In this region a read miss in L2 causes a block to be transferred from L3 to L2
  - Followed by some number of hits on the block loaded into L2
    - Depends on the stride
- As the stride increases, the ratio of L2 hits to L2 misses Increases
- Since misses are slower than hits, the read throughput Decreases
- Once stride reaches 8, Every read request misses in L2



The Memory Mountain

- To summarize:
  - Performance of the memory system is not characterized by a single number
  - Instead, it is a mountain of temporal and spatial locality
    - Elevations can vary by over an order of magnitude
  - Wise programmers try to structure their programs so that they run in the peaks instead of the valleys
- Goal:
  - Exploit temporal locality so that heavily used words are fetched from L1 cache
  - Exploit spatial locality so that as many words as possible are accessed from a single L1 cache

The Memory Mountain

- Broken record:
  - Focus your attention on inner loops
    - Bulk of computations and memory accesses
  - Try to maximize spatial locality by reading objects with stride 1
  - Try to maximize temporal locality by using a data object as often as possible once it has been read from memory